

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Load the dataset
df = pd.read_csv('data_train.csv')

# Features & target
X = df.drop('label', axis=1)
y = df['label']

# Convert problematic columns to numeric, coercing errors to NaN
# Then fill NaN values with the mean of the column
for col in X.columns:
    # Check if the column contains the problematic string '-' by attempting cor
    # This is a heuristic, a more robust check would involve checking dtypes fi
    try:
        # Attempt to convert to numeric. If it fails due to '-', errors='coerce
        X[col] = pd.to_numeric(X[col], errors='coerce')
    except:
        # If pd.to_numeric itself fails for other reasons (e.g. mixed types not
        # we might need more specific handling, but for this error, the above s
        pass

# Impute NaN values after conversion (if any were created or existed)
# It's important to only impute on numerical columns.
for col in X.select_dtypes(include=['number']).columns:
    if X[col].isnull().any():
        X[col] = X[col].fillna(X[col].mean())

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	139
1	1.00	1.00	1.00	1603
accuracy			1.00	1742
macro avg	1.00	1.00	1.00	1742

weighted avg	1.00	1.00	1.00	1742
--------------	------	------	------	------

Import Libraries

```
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Metrics
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
```

2. Load Dataset

```
df = pd.read_csv("data_train.csv") # apna file name y
df.head()
```

	label	Age	Language	Sex	Marital	Has_Credit	Field	Month_of_birth	Day_of_
0	0	34	1	2	6	2	13	12	
1	0	38	1	1	5	1	10	7	
2	0	35	1	2	4	2	9	8	
3	0	27	1	1	5	2	13	7	
4	0	32	1	2	4	2	10	7	

Next steps: [Generate code with df](#) [New interactive sheet](#)

3. Data Cleaning

```
# Replace '-' with NaN
df.replace('-', np.nan, inplace=True)
```

```
# Convert numeric columns
for col in df.columns:
    df[col] = pd.to_numeric(df[col], errors='ignore')

# Fill missing values
df.fillna(df.median(numeric_only=True), inplace=True)
```

```
/tmp/ipykernel_14911/2254715819.py:6: FutureWarning: errors='ignore' is deprecate
df[col] = pd.to_numeric(df[col], errors='ignore')
```

4. Encode Categorical Data

```
le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
```

5. Feature & Target Split

```
X = df.drop('label', axis=1)
y = df['label']
```

6. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

7. Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Logistic Regression

```
lr = LogisticRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
y_prob_lr = lr.predict_proba(X_test)[: ,1]
```

Decision Tree

```
dt = DecisionTreeClassifier(max_depth=5)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
y_prob_dt = dt.predict_proba(X_test)[:,:1]
```

Random Forest

```
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:,:1]
```

```
def evaluate_model(y_test, y_pred, y_prob, model_name):
    print(f"\n📌 {model_name}")
    print(classification_report(y_test, y_pred))
    print("ROC-AUC:", roc_auc_score(y_test, y_prob))
```

****Function for Metrics****

Function for Metrics

```
evaluate_model(y_test, y_pred_lr, y_prob_lr, "Logistic Regression")
evaluate_model(y_test, y_pred_dt, y_prob_dt, "Decision Tree")
evaluate_model(y_test, y_pred_rf, y_prob_rf, "Random Forest")
```

📌 Logistic Regression

	precision	recall	f1-score	support
0	0.94	0.99	0.96	134
1	1.00	0.99	1.00	1608
accuracy			0.99	1742
macro avg	0.97	0.99	0.98	1742
weighted avg	0.99	0.99	0.99	1742

ROC-AUC: 0.9994709289374026

📌 Decision Tree

	precision	recall	f1-score	support
0	0.99	0.98	0.98	134
1	1.00	1.00	1.00	1608

accuracy			1.00	1742
macro avg	1.00	0.99	0.99	1742
weighted avg	1.00	1.00	1.00	1742

ROC-AUC: 0.988488063414272

✂ Random Forest

	precision	recall	f1-score	support
0	1.00	0.99	0.99	134
1	1.00	1.00	1.00	1608

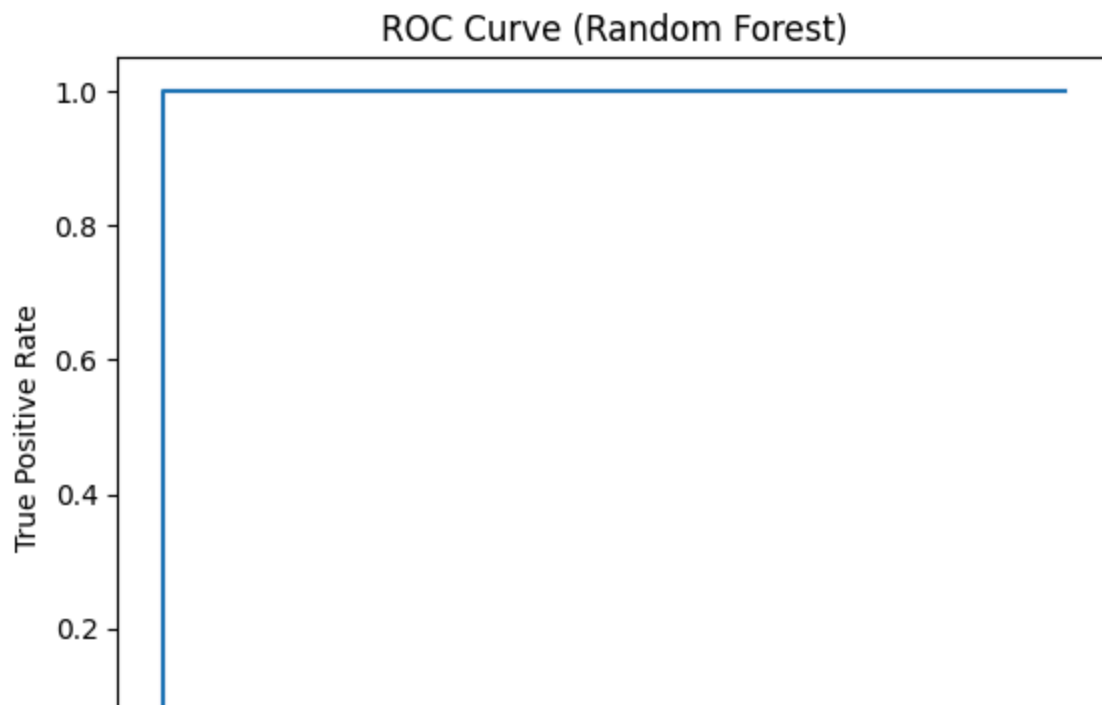
accuracy			1.00	1742
macro avg	1.00	0.99	1.00	1742
weighted avg	1.00	1.00	1.00	1742

ROC-AUC: 0.9999953590257666

ROC Curve

```
fpr, tpr, _ = roc_curve(y_test, y_prob_rf)
```

```
plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (Random Forest)")
plt.show()
```



Feature Importance

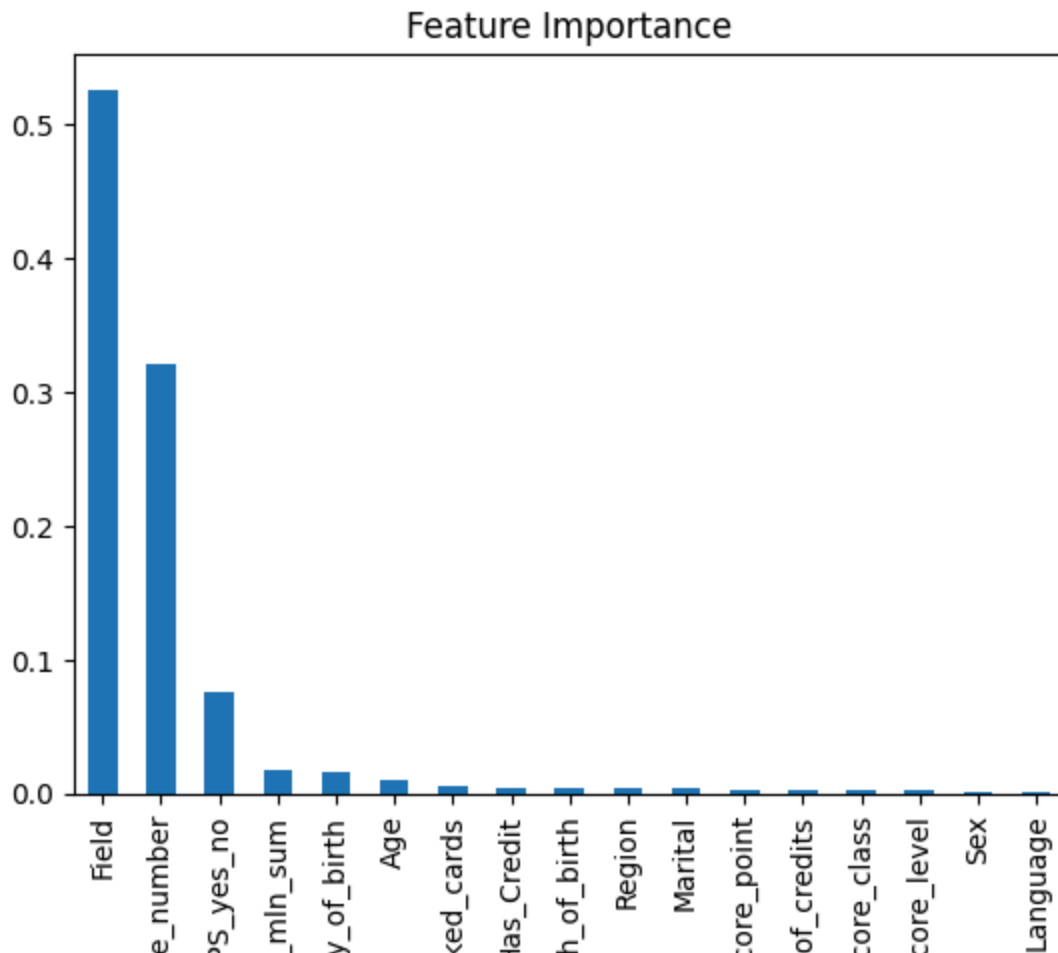
```

importances = rf.feature_importances_

feature_names = X.columns
feat_imp = pd.Series(importances, index=feature_names).sort_values(ascending=False)

feat_imp.plot(kind='bar')
plt.title("Feature Importance")
plt.show()

```



Feature Engineering Idea

```

# Example: ratio feature
df['credit_per_card'] = df['Number_of_credits'] / (df['Linked_cards'] + 1)

```

